

Implementação com DBAPI + PostgreSQL

QXD0011 - Fundamentos de Bancos de Dados

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro

victorpinheiro@ufc.br



Agenda

- Pré-requisitos
- Estrutura do Projeto
- db.py (conexão simples)
- Departamento (crud_departamento.py)
- main.py (iniciando FastAPI e incluindo rotas)

Pré-requisitos

- Instale os pacotes:
 - `pip install fastapi uvicorn psychopg2-binary`
- Supondo que seu banco já foi criado (ex: empresa) e as tabelas já existem.

Estrutura do Projeto

- empresa_api/
 - |
 - |— main.py
 - |— db.py
 - |— crud_departamento.py
 - |— crud_funcionario.py
 - |— crud_localizacao.py
 - |— crud_projeto.py
 - |— crud_trabalha_em.py
 - |— crud_dependente.py

db.py (conexão simples)

```
import psycopg2

def get_connection():
    return psycopg2.connect(
        dbname="empresa",
        user="postgres",
        password="2023",
        host="localhost"
    )
```

Departamento (crud_departamento.py)

```
router = APIRouter()

@router.get("/departamentos", response_model=List[Departamento])
async def listar_departamentos():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT dnumero, dnome, cpf_gerente, data_inicio_gerente FROM departamento")
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return [
        Departamento(
            dnumero=d[0], dnome=d[1], cpf_gerente=d[2], data_inicio_gerente=d[3]
        ) for d in rows
    ]
```

Departamento (crud_departamento.py)

```
@router.get("/departamento/{dnumero}", response_model=Departamento)
async def get_departamento(dnumero: int):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT dnumero, dnome, cpf_gerente, data_inicio_gerente FROM departamento WHERE dnumero=%s",
(dnumero,))
    row = cur.fetchone()
    cur.close()
    conn.close()
    if row:
        return Departamento(dnumero=row[0], dnome=row[1], cpf_gerente=row[2], data_inicio_gerente=row[3])
    raise HTTPException(404, "Departamento não encontrado")
```

Departamento (crud_departamento.py)

```
@router.post("/departamento")
async def criar_departamento(dep: Departamento):
    conn = get_connection()
    cur = conn.cursor()
    try:
        cur.execute(
            "INSERT INTO departamento (dnumero, dnome, cpf_gerente, data_inicio_gerente) VALUES (%s, %s, %s, %s)" ,
            (dep.dnumero, dep.dnome, dep.cpf_gerente, dep.data_inicio_gerente)
        )
        conn.commit()
    except Exception as e:
        conn.rollback()
        raise HTTPException(400, f"Erro ao criar departamento: {e}")
    finally:
        cur.close()
        conn.close()
    return {"msg": "Departamento criado com sucesso" }
```


Departamento (crud_departamento.py)

```
@router.patch("/departamento/{dnumero}")
async def atualizar_departamento_parcial(dnumero: int, dep:
DepartamentoUpdate):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT dnumero FROM departamento WHERE dnumero=%s",
(dnumero,))
    if not cur.fetchone():
        cur.close()
        conn.close()
        raise HTTPException(404, "Departamento não encontrado")
    fields = []
    values = []
    for campo, valor in dep.dict(exclude_unset=True).items():
        fields.append(f"{campo}=%s")
        values.append(valor)
```

```
if not fields:
    cur.close()
    conn.close()
    raise HTTPException(400, "Nenhum campo informado para atualização")
values.append(dnumero)
try:
    cur.execute(f"UPDATE departamento SET{'', ' '.join(fields)} WHERE
dnumero=%s", values)
    conn.commit()
except Exception as e:
    conn.rollback()
    raise HTTPException(400, f"Erro ao atualizar: {e}")
finally:
    cur.close()
    conn.close()
return {"msg": "Departamento atualizado"}
```

Departamento (crud_departamento.py)

```
@router.delete("/departamento/{dnumero}")
async def deletar_departamento(dnumero: int):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("DELETE FROM departamento WHERE dnumero=%s", (dnumero,))
    conn.commit()
    cur.close()
    conn.close()
    return {"msg": "Departamento removido"}
```

main.py (iniciando FastAPI e incluindo rotas)

```
from fastapi import FastAPI
from crud_departamento import router as departamento_router
from crud_funcionario import router as funcionario_router

app = FastAPI(
    title="API Empresa",
    version="1.0"
)
app.include_router(departamento_router)
app.include_router(funcionario_router)

# Rodar: uvicorn main:app --reload
```

API Empresa

1.0

OAS 3.1

[/openapi.json](#)

Departamentos

GET /departamentos/departamentos Listar Departamentos

GET /departamentos/departamento/{dnumero} Get Departamento

DELETE /departamentos/departamento/{dnumero} Deletar Departamento

PATCH /departamentos/departamento/{dnumero} Atualizar Departamento Parcial

POST /departamentos/departamento Criar Departamento

Bibliografia Básica

- <https://fastapi.tiangolo.com/>
- <https://fastapi.tiangolo.com/tutorial/>
- ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados. 6 ed. Pearson/Addison-Wesley, 2011. ISBN: 9788579360855



Obrigado! Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

